

Namespace F23.StringSimilarity

Classes

[Cosine](#)

[Damerau](#)

Implementation of Damerau-Levenshtein distance with transposition (also sometimes calls unrestricted Damerau-Levenshtein distance). It is the minimum number of operations needed to transform one string into the other, where an operation is defined as an insertion, deletion, or substitution of a single character, or a transposition of two adjacent characters. It does respect triangle inequality, and is thus a metric distance. This is not to be confused with the optimal string alignment distance, which is an extension where no substring can be edited more than once.

[Jaccard](#)

Each input string is converted into a set of n-grams, the Jaccard index is then computed as $|V1 \cap V2| / |V1 \cup V2|$. Like Q-Gram distance, the input strings are first converted into sets of n-grams (sequences of n characters, also called k-shingles), but this time the cardinality of each n-gram is not taken into account. Distance is computed as $1 - \text{cosine similarity}$. Jaccard index is a metric distance.

[JaroWinkler](#)

The Jaro–Winkler distance metric is designed and best suited for short strings such as person names, and to detect typos; it is (roughly) a variation of Damerau-Levenshtein, where the substitution of 2 close characters is considered less important than the substitution of 2 characters that are far from each other. Jaro-Winkler was developed in the area of record linkage (duplicate detection) (Winkler, 1990). It returns a value in the interval $[0.0, 1.0]$. The distance is computed as $1 - \text{Jaro-Winkler similarity}$.

[Levenshtein](#)

The Levenshtein distance between two words is the Minimum number of single-character edits (insertions, deletions or substitutions) required to change one string into the other.

[LongestCommonSubsequence](#)

The longest common subsequence (LCS) problem consists in finding the longest subsequence common to two (or more) sequences. It differs from problems of finding common substrings: unlike substrings, subsequences are not required to occupy consecutive positions within the original sequences.

It is used by the diff utility, by Git for reconciling multiple changes, etc.

The LCS distance between Strings X (length n) and Y (length m) is $n + m - 2 \cdot |\text{LCS}(X, Y)|$ min = 0 max = $n + m$

LCS distance is equivalent to Levenshtein distance, when only insertion and deletion is allowed (no substitution), or when the cost of the substitution is the double of the cost of an insertion or deletion.

! This class currently implements the dynamic programming approach, which has a space requirement $O(m * n)$!

[MetricLCS](#)

Distance metric based on Longest Common Subsequence, from the notes "An LCS-based string metric" by Daniel Bakkeland.

[NGram](#)

N-Gram Similarity as defined by Kondrak, "N-Gram Similarity and Distance", String Processing and Information Retrieval, Lecture Notes in Computer Science Volume 3772, 2005, pp 115-126.

The algorithm uses affixing with special character '\n' to increase the weight of first characters. The normalization is achieved by dividing the total similarity score the original length of the longest word.

total similarity score the original length of the longest word.

[NormalizedLevenshtein](#)

This distance is computed as levenshtein distance divided by the length of the longest string. The resulting value is always in the interval [0.0 1.0] but it is not a metric anymore! The similarity is computed as $1 - \text{normalized distance}$.

[OptimalStringAlignment](#)

Provides an implementation of the Optimal String Alignment (OSA) distance algorithm, which calculates the minimum number of operations required to transform one string into another. Supported operations include insertion, deletion, substitution of a single character, and transposition of two adjacent characters, with the constraint that no substring is edited more than once.

[QGram](#)

Q-gram distance, as defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches". The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrences of each n-gram): $\text{SUM}(|V1_i - V2_i|)$. Q-gram distance is a lower bound on Levenshtein distance, but can be computed in $O(m + n)$, where Levenshtein requires $O(m.n)$.

[RatcliffObershelp](#)

Ratcliff/Obershelp pattern recognition

The Ratcliff/Obershelp algorithm computes the similarity of two strings as the doubled number of matching characters divided by the total number of characters in the two strings. Matching characters are those in the longest common subsequence plus, recursively, matching characters in the unmatched region on either side of the longest common subsequence. The Ratcliff/Obershelp distance is computed as $1 - \text{Ratcliff/Obershelp similarity}$.

Author: Ligi <https://github.com/dxpux> (as a patch for fuzzystring) Ported to java from .net by denmase Ported back to .NET by paulirwin to retain compatibility with upstream Java project

[ShingleBased](#)

Base class for shingle based algorithms.

[SorensenDice](#)

Similar to Jaccard index, but this time the similarity is computed as $2 * |V1 \cap V2| / (|V1| + |V2|)$. Distance is computed as $1 - \text{cosine similarity}$.

[WeightedLevenshtein](#)

Implementation of Levenshtein that allows to define different weights for different character substitutions.

Interfaces

[ICharacterInsDel](#)

As an adjunct to [ICharacterSubstitution](#), this interface allows you to specify the cost of deletion or insertion of a character.

[ICharacterSubstitution](#)

Used to indicate the cost of character substitution.

Cost should always be in [0.0 .. 1.0] For example, in an OCR application, $\text{cost}('o', 'a')$ could be 0.4 In a checkspelling application, $\text{cost}('u', 'i')$ could be 0.4 because these are next to each other on the keyboard...

Class Cosine

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

```
public class Cosine : ShingleBased, INormalizedStringSimilarity, IStringSimilarity,
    INormalizedStringDistance, IStringDistance
```

Inheritance

[object](#)  ← [ShingleBased](#) ← Cosine

Implements

[INormalizedStringSimilarity](#), [IStringSimilarity](#), [INormalizedStringDistance](#), [IStringDistance](#)

Inherited Members

[ShingleBased.k](#) , [ShingleBased.GetProfile\(string\)](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

Cosine()

Implements Cosine Similarity between strings. The strings are first transformed in vectors of occurrences of k-shingles(sequences of k characters). In this n-dimensional space, the similarity between the two strings is the cosine of their respective vectors.

Default k is 3.

```
public Cosine()
```

Cosine(int)

Implements Cosine Similarity between strings. The strings are first transformed in vectors of occurrences of k-shingles(sequences of k characters). In this n-dimensional space, the similarity between the two strings is the cosine of their respective vectors.

```
public Cosine(int k)
```

Parameters

k [int](#)

Methods

Distance(string, string)

Returns 1.0 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1.0 - the cosine similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(IDictionary<string, int>, IDictionary<string, int>)

```
public double Similarity(IDictionary<string, int> profile1, IDictionary<string, int> profile2)
```

Parameters

profile1 [IDictionary](#) <[string](#), [int](#)>

profile2 [IDictionary](#) <[string](#), [int](#)>

Returns

[double](#)

Similarity(string, string)

Compute the cosine similarity between strings.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The cosine similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Class Damerau

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Implementation of Damerau-Levenshtein distance with transposition (also sometimes calls unrestricted Damerau-Levenshtein distance). It is the minimum number of operations needed to transform one string into the other, where an operation is defined as an insertion, deletion, or substitution of a single character, or a transposition of two adjacent characters. It does respect triangle inequality, and is thus a metric distance. This is not to be confused with the optimal string alignment distance, which is an extension where no substring can be edited more than once.

```
public class Damerau : IMetricStringDistance, IStringDistance,  
    IMetricSpanDistance, ISpanDistance
```

Inheritance

[object](#)  ← Damerau

Implements

[IMetricStringDistance](#), [IStringDistance](#), [IMetricSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

Distance(string, string)

Compute the distance between strings: the minimum number of operations needed to transform one string into the other(insertion, deletion, substitution of a single character, or a transposition of two adjacent characters).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The computed distance.

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the Damerau-Levenshtein distance between two sequences.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first sequence to compare. Cannot be [null](#).

s2 [ReadOnlySpan](#)<T>

The second sequence to compare. Cannot be [null](#).

Returns

[double](#)

The Damerau-Levenshtein distance between the two sequences, which represents the minimum number of operations (insertions, deletions, substitutions, or transpositions) required to transform one sequence into the other. Returns 0 if the sequences are equal.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#).

Remarks

The Damerau-Levenshtein distance is a metric for measuring the edit distance between two sequences, allowing for the following operations:

- Insertion of a single element.
- Deletion of a single element.
- Substitution of one element for another.
- Transposition of two adjacent elements.

This method is case-sensitive for sequences of strings or characters.

Exceptions

[ArgumentNullException](#)

Thrown if **s1** or **s2** is [null](#).

Interface ICharacterInsDel

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

As an adjunct to [ICharacterSubstitution](#), this interface allows you to specify the cost of deletion or insertion of a character.

```
public interface ICharacterInsDel
```

Methods

DeletionCost(char)

Computes the deletion cost.

```
double DeletionCost(char c)
```

Parameters

c [char](#) 

The character being deleted.

Returns

[double](#) 

The cost to be allocated to deleting the given character, in the range [0, 1].

InsertionCost(char)

Computes the insertion cost.

```
double InsertionCost(char c)
```

Parameters

c [char](#)

The character being inserted.

Returns

[double](#)

The cost to be allocated to inserting the given character, in the range [0, 1].

Interface ICharacterSubstitution

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Used to indicate the cost of character substitution.

Cost should always be in [0.0 .. 1.0] For example, in an OCR application, cost('o', 'a') could be 0.4 In a checkspelling application, cost('u', 'i') could be 0.4 because these are next to each other on the keyboard...

```
public interface ICharacterSubstitution
```

Methods

Cost(char, char)

Indicate the cost of substitution c1 and c2.

```
double Cost(char c1, char c2)
```

Parameters

c1 [char](#)

The first character of the substitution.

c2 [char](#)

The second character of the substitution.

Returns

[double](#)

The cost in the range [0, 1].

Class Jaccard

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Each input string is converted into a set of n-grams, the Jaccard index is then computed as $|V1 \cap V2| / |V1 \cup V2|$. Like Q-Gram distance, the input strings are first converted into sets of n-grams (sequences of n characters, also called k-shingles), but this time the cardinality of each n-gram is not taken into account. Distance is computed as 1 - cosine similarity. Jaccard index is a metric distance.

```
public class Jaccard : ShingleBased, IMetricStringDistance, INormalizedStringDistance,
    IStringDistance, INormalizedStringSimilarity, IStringSimilarity
```

Inheritance

[object](#)  ← [ShingleBased](#) ← Jaccard

Implements

[IMetricStringDistance](#), [INormalizedStringDistance](#), [IStringDistance](#), [INormalizedStringSimilarity](#), [IStringSimilarity](#)

Inherited Members

[ShingleBased.k](#), [ShingleBased.GetProfile\(string\)](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Jaccard()

The strings are first transformed into sets of k-shingles (sequences of k characters), then Jaccard index is computed as $|A \cap B| / |A \cup B|$. The default value of k is 3.

```
public Jaccard()
```

Jaccard(int)

The strings are first transformed into sets of k-shingles (sequences of k characters), then Jaccard index is computed as $|A \cap B| / |A \cup B|$. The default value of k is 3.

```
public Jaccard(int k)
```

Parameters

k [int](#)

Methods

Distance(string, string)

Distance is computed as 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - the Jaccard similarity.

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(string, string)

Compute jaccard index: $|A \cap B| / |A \cup B|$.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#) 

The second string to compare.

Returns

[double](#) 

The Jaccard index in the range [0, 1]

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class JaroWinkler

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

The Jaro–Winkler distance metric is designed and best suited for short strings such as person names, and to detect typos; it is (roughly) a variation of Damerau-Levenshtein, where the substitution of 2 close characters is considered less important than the substitution of 2 characters that are far from each other. Jaro-Winkler was developed in the area of record linkage (duplicate detection) (Winkler, 1990). It returns a value in the interval [0.0, 1.0]. The distance is computed as 1 - Jaro-Winkler similarity.

```
public class JaroWinkler : INormalizedStringSimilarity, IStringSimilarity,
    INormalizedStringDistance, IStringDistance, INormalizedSpanSimilarity, ISpanSimilarity,
    INormalizedSpanDistance, ISpanDistance
```

Inheritance

[object](#) ← JaroWinkler

Implements

[INormalizedStringSimilarity](#), [IStringSimilarity](#), [INormalizedStringDistance](#), [IStringDistance](#),
[INormalizedSpanSimilarity](#), [ISpanSimilarity](#), [INormalizedSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

JaroWinkler()

Creates a new instance with default threshold (0.7)

```
public JaroWinkler()
```

JaroWinkler(double)

Creates a new instance with given threshold to determine when Winkler bonus should be used. Set threshold to a negative value to get the Jaro distance.

```
public JaroWinkler(double threshold)
```

Parameters

threshold [double](#)

Methods

Distance(string, string)

Return 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - similarity

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the distance between two sequences based on their similarity.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan<T>](#)

The first sequence to compare.

s2 [ReadOnlySpan<T>](#)

The second sequence to compare.

Returns

[double](#)

A double value representing the distance between the two sequences. The value ranges from 0.0 to 1.0, where 0.0 indicates identical sequences and 1.0 indicates completely dissimilar sequences.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#).

Remarks

The distance is calculated as the complement of the similarity between the two sequences.

Similarity(string, string)

Compute Jaro-Winkler similarity.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The Jaro-Winkler similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the similarity between two sequences using the Jaro-Winkler distance metric.

```
public double Similarity<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first sequence to compare. Cannot be null.

s2 [ReadOnlySpan](#)<T>

The second sequence to compare. Cannot be null.

Returns

[double](#)

A value between 0 and 1 representing the similarity between the two sequences, where 1 indicates identical sequences and 0 indicates no similarity.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#) .

Remarks

The similarity is calculated using the Jaro-Winkler distance, which is a measure of similarity between two sequences. The result is adjusted based on common prefixes to give higher scores to sequences that share a common prefix.

Exceptions

[ArgumentNullException](#) 

Thrown if **s1** or **s2** is null.

Class Levenshtein

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

The Levenshtein distance between two words is the Minimum number of single-character edits (insertions, deletions or substitutions) required to change one string into the other.

```
public class Levenshtein : IMetricStringDistance, IStringDistance,
    IMetricSpanDistance, ISpanDistance
```

Inheritance

[object](#) ← Levenshtein

Implements

[IMetricStringDistance](#), [IStringDistance](#), [IMetricSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Equivalent to Distance(s1, s2, Int32.MaxValue).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The Levenshtein distance between strings

Distance(string, string, int)

The Levenshtein distance, or edit distance, between two words is the Minimum number of single-character edits (insertions, deletions or substitutions) required to change one word into the other.

http://en.wikipedia.org/wiki/Levenshtein_distance

It is always at least the difference of the sizes of the two strings. It is at most the length of the longer string. It is zero if and only if the strings are equal. If the strings are the same size, the HamMing distance is an upper bound on the Levenshtein distance. The Levenshtein distance verifies the triangle inequality (the distance between two strings is no greater than the sum Levenshtein distances from a third string).

Implementation uses dynamic programMing (Wagner–Fischer algorithm), with only 2 rows of data. The space requirement is thus $O(m)$ and the algorithm runs in $O(mn)$.

```
public double Distance(string s1, string s2, int limit)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

limit [int](#)

The maximum result to compute before stopping. This means that the calculation can terminate early if you only care about strings with a certain similarity. Set this to `Int32.MaxValue` if you want to run the calculation to completion in every case.

Returns

[double](#)

The Levenshtein distance between strings

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the distance between two sequences of elements.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)  <T>

The first sequence to compare.

s2 [ReadOnlySpan](#)  <T>

The second sequence to compare.

Returns

[double](#) 

A [double](#)  representing the distance between the two sequences. The specific meaning of the distance depends on the implementation of the comparison logic.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#) .

Remarks

This method uses a default maximum threshold for the distance calculation. For custom thresholds, use an overload that accepts a threshold parameter.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>, int)

Calculates the edit distance (Levenshtein distance) between two sequences, with an optional upper limit.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2, int limit) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan<T>](#)

The first sequence to compare. Cannot be null.

s2 [ReadOnlySpan<T>](#)

The second sequence to compare. Cannot be null.

limit [int](#)

The maximum distance to calculate. If the edit distance exceeds this value, the method returns **limit**.

Returns

[double](#)

The edit distance between **s1** and **s2**. If the sequences are identical, the result is 0. If the distance exceeds **limit**, the method returns **limit**.

Type Parameters

T

The type of elements in the sequences. The type must implement [IEquatable<T>](#).

Remarks

The edit distance is a measure of the minimum number of single-element edits (insertions, deletions, or substitutions) required to transform one sequence into the other. This method is optimized to stop processing early if the distance exceeds the specified **limit**.

Exceptions

[ArgumentNullException](#)

Thrown if `s1` or `s2` is null.

Class LongestCommonSubsequence

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

The longest common subsequence (LCS) problem consists in finding the longest subsequence common to two (or more) sequences. It differs from problems of finding common substrings: unlike substrings, subsequences are not required to occupy consecutive positions within the original sequences.

It is used by the diff utility, by Git for reconciling multiple changes, etc.

The LCS distance between Strings X (length n) and Y (length m) is $n + m - 2 |LCS(X, Y)|$ min = 0 max = n + m

LCS distance is equivalent to Levenshtein distance, when only insertion and deletion is allowed (no substitution), or when the cost of the substitution is the double of the cost of an insertion or deletion.

! This class currently implements the dynamic programming approach, which has a space requirement $O(m * n)$!

```
public class LongestCommonSubsequence : IStringDistance, ISpanDistance
```

Inheritance

[object](#) ← LongestCommonSubsequence

Implements

[IStringDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Return the LCS distance between strings s1 and s2, computed as $|s1| + |s2| - 2 * |LCS(s1, s2)|$.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The LCS distance between strings s1 and s2, computed as $|s1| + |s2| - 2 * |LCS(s1, s2)|$

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the distance between two sequences based on their similarity.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first sequence to compare. Cannot be empty or null.

s2 [ReadOnlySpan](#)<T>

The second sequence to compare. Cannot be empty or null.

Returns

[double](#)

A non-negative [double](#) representing the distance between the two sequences. Returns 0 if the sequences are identical.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#).

Remarks

The distance is calculated as the sum of the lengths of the two sequences minus twice the length of their longest common subsequence.

Exceptions

[ArgumentNullException](#)

Thrown if `s1` or `s2` is [null](#).

Length(string, string)

Return the length of Longest Common Subsequence (LCS) between strings `s1` and `s2`.

```
public int Length(string s1, string s2)
```

Parameters

`s1` [string](#)

The first string to compare.

`s2` [string](#)

The second string to compare.

Returns

[int](#)

The length of LCS(`s2`, `s2`)

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class MetricLCS

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Distance metric based on Longest Common Subsequence, from the notes "An LCS-based string metric" by Daniel Bakkeland.

```
public class MetricLCS : IMetricStringDistance, INormalizedStringDistance, IStringDistance,
    IMetricSpanDistance, ISpanDistance
```

Inheritance

[object](#) ← MetricLCS

Implements

[IMetricStringDistance](#), [INormalizedStringDistance](#), [IStringDistance](#), [IMetricSpanDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Distance metric based on Longest Common Subsequence, computed as $1 - |\text{LCS}(s1, s2)| / \max(|s1|, |s2|)$.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

LCS distance metric

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the normalized distance between two sequences based on their longest common subsequence.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first sequence to compare. Cannot be null.

s2 [ReadOnlySpan](#)<T>

The second sequence to compare. Cannot be null.

Returns

[double](#)

A value between 0.0 and 1.0 representing the normalized distance between the two sequences:

- Returns 0.0 if the sequences are identical.
- Returns 1.0 if the sequences have no common elements.
- Returns a value between 0.0 and 1.0 for partial similarity.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#).

Remarks

The distance is calculated as:

$$1.0 - (\text{Length of Longest Common Subsequence} / \text{Maximum Length of the Two Sequences})$$

This method is case-sensitive for sequences of strings or characters.

Exceptions

[ArgumentNullException](#)

Thrown if *s1* or *s2* is [null](#).

Class NGram

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

N-Gram Similarity as defined by Kondrak, "N-Gram Similarity and Distance", String Processing and Information Retrieval, Lecture Notes in Computer Science Volume 3772, 2005, pp 115-126.

The algorithm uses affixing with special character '\n' to increase the weight of first characters. The normalization is achieved by dividing the total similarity score the original length of the longest word.

total similarity score the original length of the longest word.

```
public class NGram : INormalizedStringDistance, IStringDistance
```

Inheritance

[object](#)  ← NGram

Implements

[INormalizedStringDistance](#), [IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

NGram()

Initializes a new instance of the [NGram](#) class with the default value for N.

```
public NGram()
```

Remarks

This constructor sets the N-gram size to the default value defined by F23.StringSimilarity.NGram.DEFAULT_N. Use this constructor when you want to create an NGram instance with the default configuration.

NGram(int)

Initializes a new instance of the [NGram](#) class with the specified size.

```
public NGram(int n)
```

Parameters

n [int](#)

The size of the n-gram. Must be a positive integer.

Methods

Distance(string, string)

Compute n-gram distance.

```
public double Distance(string s0, string s1)
```

Parameters

s0 [string](#)

The first string to compare.

s1 [string](#)

The second string to compare.

Returns

[double](#)

The computed n-gram distance in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s0 or s1 is null.

Class NormalizedLevenshtein

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

This distance is computed as levenshtein distance divided by the length of the longest string. The resulting value is always in the interval [0.0 1.0] but it is not a metric anymore! The similarity is computed as 1 - normalized distance.

```
public class NormalizedLevenshtein : INormalizedStringDistance, IStringDistance,
    INormalizedStringSimilarity, IStringSimilarity, INormalizedSpanDistance, ISpanDistance,
    INormalizedSpanSimilarity, ISpanSimilarity
```

Inheritance

[object](#) ← NormalizedLevenshtein

Implements

[INormalizedStringDistance](#), [IStringDistance](#), [INormalizedStringSimilarity](#), [IStringSimilarity](#),
[INormalizedSpanDistance](#), [ISpanDistance](#), [INormalizedSpanSimilarity](#), [ISpanSimilarity](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Compute distance as Levenshtein(s1, s2) / max(|s1|, |s2|).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The computed distance in the range [0, 1]

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the normalized distance between two sequences of elements.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)<T>

The first sequence to compare. Cannot be empty or null.

s2 [ReadOnlySpan](#)<T>

The second sequence to compare. Cannot be empty or null.

Returns

[double](#)

A double value representing the normalized distance between the two sequences. Returns 0.0 if the sequences are equal or both are empty.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#).

Remarks

The distance is normalized by the length of the longer sequence. This ensures the result is always in the range [0.0, 1.0], where 0.0 indicates identical sequences and 1.0 indicates maximum dissimilarity.

Exceptions

[ArgumentNullException](#) 

Thrown if `s1` or `s2` is null.

Similarity(string, string)

Return 1 - distance.

```
public double Similarity(string s1, string s2)
```

Parameters

`s1` [string](#) 

The first string to compare.

`s2` [string](#) 

The second string to compare.

Returns

[double](#) 

1 - distance

Exceptions

[ArgumentNullException](#) 

If `s1` or `s2` is null.

Similarity<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the similarity between two sequences based on their distance.

```
public double Similarity<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan<T>](#)

The first sequence to compare.

s2 [ReadOnlySpan<T>](#)

The second sequence to compare.

Returns

[double](#)

A value between 0.0 and 1.0 representing the similarity of the two sequences, where 1.0 indicates identical sequences and 0.0 indicates completely dissimilar sequences.

Type Parameters

T

The type of elements in the sequences. Must implement [IEquatable<T>](#).

Remarks

The similarity is calculated as 1.0 minus the distance between the two sequences.

Class OptimalStringAlignment

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Provides an implementation of the Optimal String Alignment (OSA) distance algorithm, which calculates the minimum number of operations required to transform one string into another. Supported operations include insertion, deletion, substitution of a single character, and transposition of two adjacent characters, with the constraint that no substring is edited more than once.

```
public sealed class OptimalStringAlignment : IStringDistance, ISpanDistance
```

Inheritance

[object](#) ← OptimalStringAlignment

Implements

[IStringDistance](#), [ISpanDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Remarks

This class is designed for use in scenarios where a measure of similarity or difference between two strings or spans is required. It supports both string and span-based inputs, making it suitable for high-performance applications where memory efficiency is important. The OSA distance is particularly useful in applications such as spell checking, approximate string matching, and natural language processing, where transpositions (e.g., swapping two adjacent characters) are common errors.

Methods

Distance(string, string)

Compute the distance between strings: the minimum number of operations needed to transform one string into the other (insertion, deletion, substitution of a single character, or a transposition of two adjacent characters) while no substring is edited more than once.

```
public double Distance(string s1, string s2)
```


Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#) 

The second string to compare.

Returns

[double](#) 

the OSA distance

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Calculates the Damerau-Levenshtein distance between two sequences.

```
public double Distance<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan](#)  <T>

The first sequence to compare. Cannot be null.

s2 [ReadOnlySpan](#)  <T>

The second sequence to compare. Cannot be null.

Returns

[double](#) 

The Damerau-Levenshtein distance between the two sequences, which represents the minimum number of single-character edits (insertions, deletions, substitutions, or transpositions) required to transform one sequence into the other.

Type Parameters

T

The type of elements in the sequences. The type must implement [IEquatable<T>](#).

Remarks

The Damerau-Levenshtein distance is a metric for measuring the difference between two sequences. It extends the Levenshtein distance by allowing transpositions of adjacent characters as a single edit operation. This method is case-sensitive for sequences of characters.

Exceptions

[ArgumentNullException](#)

Thrown if **s1** or **s2** is null.

Class QGram

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Q-gram distance, as defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches". The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrences of each n-gram): $\text{SUM}(|V1_i - V2_i|)$. Q-gram distance is a lower bound on Levenshtein distance, but can be computed in $O(m + n)$, where Levenshtein requires $O(m.n)$.

```
public class QGram : ShingleBased, IStringDistance
```

Inheritance

[object](#)  ← [ShingleBased](#) ← QGram

Implements

[IStringDistance](#)

Inherited Members

[ShingleBased.k](#) , [ShingleBased.GetProfile\(string\)](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

QGram()

Q-gram similarity and distance. Defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches", <http://www.sciencedirect.com/science/article/pii/0304397592901434>  The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrence of each k-shingle). Q-gram distance is a lower bound on Levenshtein distance, but can be computed in $O(|A| + |B|)$, where Levenshtein requires $O(|A|.|B|)$ Default k is 3.

```
public QGram()
```

QGram(int)

Q-gram similarity and distance. Defined by Ukkonen in "Approximate string-matching with q-grams and maximal matches", <http://www.sciencedirect.com/science/article/pii/0304397592901434> The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrences of each k-shingle). Q-gram distance is a lower bound on Levenshtein distance, but can be computed in $O(|A| + |B|)$, where Levenshtein requires $O(|A|.|B|)$

```
public QGram(int k)
```

Parameters

k [int](#)

Methods

Distance(IDictionary<string, int>, IDictionary<string, int>)

Compute QGram distance using precomputed profiles.

```
public double Distance(IDictionary<string, int> profile1, IDictionary<string, int> profile2)
```

Parameters

profile1 [IDictionary](#) <[string](#), [int](#)>

profile2 [IDictionary](#) <[string](#), [int](#)>

Returns

[double](#)

Distance(string, string)

The distance between two strings is defined as the L1 norm of the difference of their profiles (the number of occurrence of each k-shingle).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string to compare.

s2 [string](#) 

The second string to compare.

Returns

[double](#) 

The computed Q-gram distance.

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class RatcliffObershelp

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Ratcliff/Obershelp pattern recognition

The Ratcliff/Obershelp algorithm computes the similarity of two strings as the doubled number of matching characters divided by the total number of characters in the two strings. Matching characters are those in the longest common subsequence plus, recursively, matching characters in the unmatched region on either side of the longest common subsequence. The Ratcliff/Obershelp distance is computed as 1 - Ratcliff/Obershelp similarity.

Author: Ligi <https://github.com/dxpux> (as a patch for fuzzystring) Ported to java from .net by denmase
Ported back to .NET by paulirwin to retain compatibility with upstream Java project

```
public class RatcliffObershelp : INormalizedStringSimilarity, IStringSimilarity,  
    INormalizedStringDistance, IStringDistance
```

Inheritance

[object](#) ← RatcliffObershelp

Implements

[INormalizedStringSimilarity](#), [IStringSimilarity](#), [INormalizedStringDistance](#), [IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#),
[object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

Distance(string, string)

Return 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1 - similarity

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(string, string)

Compute the Ratcliff-Obershelp similarity between strings.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The RatcliffObershelp similarity in the range [0, 1]

Exceptions

[ArgumentNullException](#) 

If s1 or s2 is null.

Class ShingleBased

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Base class for shingle based algorithms.

```
public abstract class ShingleBased
```

Inheritance

[object](#)  ← ShingleBased

Derived

[Cosine](#), [Jaccard](#), [QGram](#), [SorensenDice](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

ShingleBased()

Initializes a new instance of the [ShingleBased](#) class with the default shingle size.

```
protected ShingleBased()
```

ShingleBased(int)

```
protected ShingleBased(int k)
```

Parameters

k [int](#) 

Exceptions

If k is less than or equal to 0.

Properties

k

Return k, the length of k-shingles (aka n-grams).

```
protected int k { get; }
```

Property Value

[int](#)

Methods

GetProfile(string)

Generates a profile of k-length substrings (shingles) from the specified string, along with their frequency of occurrence.

```
public Dictionary<string, int> GetProfile(string s)
```

Parameters

s [string](#)

The input string from which to generate the shingle profile. Cannot be null.

Returns

[Dictionary](#) <[string](#), [int](#)>

A dictionary where the keys are k-length substrings (shingles) extracted from the input string, and the values are the number of times each shingle appears.

Remarks

This method processes the input string by normalizing spaces and then extracting overlapping substrings of length k . The resulting dictionary provides a frequency count for each unique shingle.

Class SorensenDice

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Similar to Jaccard index, but this time the similarity is computed as $2 * |V1 \text{ inter } V2| / (|V1| + |V2|)$. Distance is computed as $1 - \text{cosine similarity}$.

```
public class SorensenDice : ShingleBased, INormalizedStringDistance, IStringDistance,
    INormalizedStringSimilarity, IStringSimilarity
```

Inheritance

[object](#)  ← [ShingleBased](#) ← SorensenDice

Implements

[INormalizedStringDistance](#), [IStringDistance](#), [INormalizedStringSimilarity](#), [IStringSimilarity](#)

Inherited Members

[ShingleBased.k](#), [ShingleBased.GetProfile\(string\)](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

SorensenDice()

Sorensen-Dice coefficient, aka Sørensen index, Dice's coefficient or Czekanowski's binary (non-quantitative) index.

The strings are first converted to boolean sets of k-shingles (sequences of k characters), then the similarity is computed as $2 * |A \text{ inter } B| / (|A| + |B|)$. Attention: Sorensen-Dice distance (and similarity) does not satisfy triangle inequality. Default k is 3.

```
public SorensenDice()
```

SorensenDice(int)

Sorensen-Dice coefficient, aka Sørensen index, Dice's coefficient or Czekanowski's binary (non-quantitative) index.

The strings are first converted to boolean sets of k-shingles (sequences of k characters), then the similarity is computed as $2 * |A \cap B| / (|A| + |B|)$. Attention: Sorensen-Dice distance (and similarity) does not satisfy triangle inequality.

```
public SorensenDice(int k)
```

Parameters

k [int](#)

Methods

Distance(string, string)

Returns 1 - similarity.

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

1.0 - the computed similarity

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Similarity(string, string)

Similarity is computed as $2 * |A \cap B| / (|A| + |B|)$.

```
public double Similarity(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The computed Sorensen-Dice similarity.

Exceptions

[ArgumentNullException](#)

If s1 or s2 is null.

Class WeightedLevenshtein

Namespace: [F23.StringSimilarity](#)

Assembly: F23.StringSimilarity.dll

Implementation of Levenshtein that allows to define different weights for different character substitutions.

```
public class WeightedLevenshtein : IStringDistance
```

Inheritance

[object](#) ← WeightedLevenshtein

Implements

[IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

WeightedLevenshtein(ICharacterSubstitution)

Instantiate with provided character substitution.

```
public WeightedLevenshtein(ICharacterSubstitution characterSubstitution)
```

Parameters

characterSubstitution [ICharacterSubstitution](#)

The strategy to determine character substitution weights.

WeightedLevenshtein(ICharacterSubstitution, ICharacterInsDel)

Instantiate with provided character substitution, insertion, and deletion weights.

```
public WeightedLevenshtein(ICharacterSubstitution characterSubstitution,  
    ICharacterInsDel characterInsDel)
```

Parameters

characterSubstitution [ICharacterSubstitution](#)

The strategy to determine character substitution weights.

characterInsDel [ICharacterInsDel](#)

The strategy to determine character insertion/deletion weights.

Methods

Distance(string, string)

Equivalent to Distance(s1, s2, Double.MaxValue).

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

Returns

[double](#)

The computed weighted Levenshtein distance.

Distance(string, string, double)

Compute Levenshtein distance using provided weights for substitution.


```
public double Distance(string s1, string s2, double limit)
```

Parameters

s1 [string](#)

The first string to compare.

s2 [string](#)

The second string to compare.

limit [double](#)

The maximum result to compute before stopping. This means that the calculation can terminate early if you only care about strings with a certain similarity. Set this to `Double.MaxValue` if you want to run the calculation to completion in every case.

Returns

[double](#)

The computed weighted Levenshtein distance.

Exceptions

[ArgumentNullException](#)

If `s1` or `s2` is null.

Namespace F23.StringSimilarity.Experimental

Classes

[Sift4](#)

Sift4 - a general purpose string distance algorithm inspired by JaroWinkler and Longest Common Subsequence. Original JavaScript algorithm by siderite, java port by Nathan Fischer 2016.

<https://siderite.dev/blog/super-fast-and-accurate-string-distance.html> 

<https://blackdoor.github.io/blog/sift4-java/> 

Class Sift4

Namespace: [F23.StringSimilarity.Experimental](#)

Assembly: F23.StringSimilarity.dll

Sift4 - a general purpose string distance algorithm inspired by JaroWinkler and Longest Common Subsequence. Original JavaScript algorithm by siderite, java port by Nathan Fischer 2016.

<https://siderite.dev/blog/super-fast-and-accurate-string-distance.html>

<https://blackdoor.github.io/blog/sift4-java/>

```
public class Sift4 : IStringDistance
```

Inheritance

[object](#) ← Sift4

Implements

[IStringDistance](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Properties

MaxOffset

Gets or sets the maximum distance to search for character transposition. Compose cost of algorithm is $O(n \cdot \text{MaxOffset})$

```
public int MaxOffset { get; set; }
```

Property Value

[int](#)

Methods

Distance(string, string)

Sift4 - a general purpose string distance algorithm inspired by JaroWinkler and Longest Common Subsequence. Original JavaScript algorithm by siderite, java port by Nathan Fischer 2016.

<https://siderite.dev/blog/super-fast-and-accurate-string-distance.html>

<https://blackdoor.github.io/blog/sift4-java/>

```
public double Distance(string s1, string s2)
```

Parameters

s1 [string](#)

s2 [string](#)

Returns

[double](#)

Namespace F23.StringSimilarity.Interfaces

Interfaces

[IMetricSpanDistance](#)

Span distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

[IMetricStringDistance](#)

String distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

[INormalizedSpanDistance](#)

An interface for normalized distance measures that operate on spans.

[INormalizedSpanSimilarity](#)

Defines a contract for calculating the similarity between spans of text, normalized to a range of 0 to 1.

[INormalizedStringDistance](#)

Interface for normalized string distance algorithms

[INormalizedStringSimilarity](#)

Interface for normalized string similarity algorithms

[ISpanDistance](#)

An interface for distance measures that operate on spans.

[ISpanSimilarity](#)

Interface for span similarity algorithms

[IStringDistance](#)

Interface for string distance algorithms

[IStringSimilarity](#)

Interface for string similarity algorithms

Interface IMetricSpanDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Span distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

```
public interface IMetricSpanDistance : ISpanDistance
```

Methods

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return the metric distance.

```
double Distance<T>(ReadOnlySpan<T> b1, ReadOnlySpan<T> b2) where T : IEquatable<T>
```

Parameters

b1 [ReadOnlySpan<T>](#)

The first span.

b2 [ReadOnlySpan<T>](#)

The second span.

Returns

[double](#)

The metric distance.

Type Parameters

Interface IMetricStringDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

String distances that implement this interface are metrics, which means:

- $d(x, y) \geq 0$ (non-negativity, or separation axiom)
- $d(x, y) = 0$ if and only if $x = y$ (identity, or coincidence axiom)
- $d(x, y) = d(y, x)$ (symmetry)
- $d(x, z) \leq d(x, y) + d(y, z)$ (triangle inequality).

```
public interface IMetricStringDistance : IStringDistance
```

Methods

Distance(string, string)

Compute and return the metric distance.

```
double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

s2 [string](#) 

Returns

[double](#) 

Interface INormalizedSpanDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

An interface for normalized distance measures that operate on spans.

```
public interface INormalizedSpanDistance : ISpanDistance
```

Inherited Members

[ISpanDistance.Distance<T>\(ReadOnlySpan<T>, ReadOnlySpan<T>\)](#).

Interface INormalizedSpanSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Defines a contract for calculating the similarity between spans of text, normalized to a range of 0 to 1.

```
public interface INormalizedSpanSimilarity : ISpanSimilarity
```

Inherited Members

[ISpanSimilarity.Similarity<T>\(ReadOnlySpan<T>, ReadOnlySpan<T>\)](#).

Remarks

This interface extends [ISpanSimilarity](#) by ensuring that similarity scores are normalized. A score of 0 indicates no similarity, while a score of 1 indicates identical spans.

Interface INormalizedStringDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Interface for normalized string distance algorithms

```
public interface INormalizedStringDistance : IStringDistance
```

Inherited Members

[IStringDistance.Distance\(string, string\)](#).

Interface INormalizedStringSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Interface for normalized string similarity algorithms

```
public interface INormalizedStringSimilarity : IStringSimilarity
```

Inherited Members

[IStringSimilarity.Similarity\(string, string\)](#)

Interface ISpanDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

An interface for distance measures that operate on spans.

```
public interface ISpanDistance
```

Methods

Distance<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of distance. Must be ≥ 0 .

This method operates on spans such as byte arrays. Note that, when used on bytes, string encodings that use more than one byte per codepoint (such as UTF-8) are not supported and will most likely return incorrect results.

```
double Distance<T>(ReadOnlySpan<T> b1, ReadOnlySpan<T> b2) where T : IEquatable<T>
```

Parameters

b1 [ReadOnlySpan](#)[↗]<T>

The first span.

b2 [ReadOnlySpan](#)[↗]<T>

The second span.

Returns

[double](#)[↗]

The measure of distance between the spans.

Type Parameters

Interface ISpanSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Interface for span similarity algorithms

```
public interface ISpanSimilarity
```

Methods

Similarity<T>(ReadOnlySpan<T>, ReadOnlySpan<T>)

Compute and return a measure of similarity between 2 spans.

```
double Similarity<T>(ReadOnlySpan<T> s1, ReadOnlySpan<T> s2) where T : IEquatable<T>
```

Parameters

s1 [ReadOnlySpan<T>](#)

The first span

s2 [ReadOnlySpan<T>](#)

The second span

Returns

[double](#)

Similarity (0 means both spans are completely different)

Type Parameters

T

Interface IStringDistance

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Interface for string distance algorithms

```
public interface IStringDistance
```

Methods

Distance(string, string)

Compute and return a measure of distance. Must be ≥ 0 .

```
double Distance(string s1, string s2)
```

Parameters

s1 [string](#) 

s2 [string](#) 

Returns

[double](#) 

Interface IStringSimilarity

Namespace: [F23.StringSimilarity.Interfaces](#)

Assembly: F23.StringSimilarity.dll

Interface for string similarity algorithms

```
public interface IStringSimilarity
```

Methods

Similarity(string, string)

Compute and return a measure of similarity between 2 strings.

```
double Similarity(string s1, string s2)
```

Parameters

s1 [string](#) 

The first string

s2 [string](#) 

The second string

Returns

[double](#) 

Similarity (0 means both strings are completely different)